

Python Programming

Day 7: NumPy 기초

ndarray, Array Creation, Indexing, Vectorized Operations, Broadcasting, Linear Algebra

Contents

1	학습 목표	2
2	NumPy와 ndarray	2
2.1	ndarray	2
2.2	최종 코드: 01_numpy_intro.py	2
3	NumPy 배열 생성	3
3.1	zeros()와 ones()	3
3.2	arange()	3
3.3	linspace()	3
3.4	최종 코드: 02_numpy_array.py	3
4	인덱싱과 슬라이싱	4
4.1	최종 코드: 03_numpy_indexing.py	4
4.2	NumPy 슬라이싱의 View	4
5	벡터화 연산	5
5.1	최종 코드: 04_numpy_operations.py	5
5.2	원소별 곱과 행렬곱	5
6	브로드캐스팅	5
6.1	행렬과 벡터의 브로드캐스팅	6
6.2	최종 코드: 05_numpy_broadcasting.py	6
7	NumPy 선형대수	6
7.1	최종 코드: 06_numpy_linear_algebra.py	7
7.2	연립방정식 풀이	7
8	Day 7 종합 실습: NumPy 기초 복습	8
8.1	프로그램 요구사항	8
8.2	최종 코드: practice_day07.py	8
8.3	실행 결과	9
8.4	프로그램 구조 분석	10
9	실습 파일 목록	10
10	핵심 요약	10

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- NumPy의 목적과 ndarray의 의미 설명하기
- `np.array()`로 NumPy 배열 생성하기
- `zeros()`, `ones()`, `arange()`, `linspace()` 사용하기
- 1차원 배열에서 인덱싱과 슬라이싱 사용하기
- 배열 사이의 원소별 연산과 스칼라 연산 수행하기
- 브로드캐스팅을 이용하여 크기가 다른 배열 계산하기
- `@`, `.T`, `det()`, `inv()`로 기본 행렬 연산 수행하기

2. NumPy와 ndarray

NumPy는 Numerical Python의 줄임말로, Python에서 수치 계산을 효율적으로 수행하기 위한 라이브러리이다. Python 리스트도 숫자를 저장할 수 있지만, NumPy 배열은 대량의 수치 데이터를 빠르게 처리하고 행렬 및 선형대수 계산을 수행하는 데 적합하다.

NumPy를 설치하지 않았다면 터미널에서 다음 명령을 한 번 실행한다.

```
pip install numpy
```

NumPy는 일반적으로 `np`라는 별칭으로 불려온다.

```
1 import numpy as np
```

2.1. ndarray

NumPy의 기본 배열 자료형은 ndarray이다. `np.array()`에 Python 리스트를 전달하면 NumPy 배열을 만들 수 있다.

2.2. 최종 코드: 01_numpy_intro.py

```
1 import numpy as np
2
3 numbers = np.array([1, 2, 3, 4, 5])
4
5 print(numbers)
6 print(type(numbers))
```

실행 결과는 다음과 같다.

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

Python 리스트는 원소 사이에 쉼표가 표시되지만, NumPy 배열은 기본 출력에서 쉼표 없이 표시된다.

핵심 포인트

NumPy의 대부분 기능은 `ndarray`를 중심으로 동작한다. 이후의 인덱싱, 벡터화 연산, 브로드캐스팅, 행렬 연산도 모두 NumPy 배열을 대상으로 수행한다.

3. NumPy 배열 생성

NumPy는 리스트를 변환하는 것뿐 아니라 일정한 규칙을 가진 배열을 빠르게 생성하는 함수도 제공한다.

함수	기능
<code>np.zeros(n)</code>	0으로 채운 길이 <code>n</code> 의 배열 생성
<code>np.ones(n)</code>	1로 채운 길이 <code>n</code> 의 배열 생성
<code>np.arange(start, stop, step)</code>	일정한 간격의 배열 생성
<code>np.linspace(start, stop, count)</code>	구간을 균등하게 나눈 배열 생성

3.1. `zeros()`와 `ones()`

`np.zeros(5)`는 0을 다섯 개 가진 배열을 만들고, `np.ones(5)`는 1을 다섯 개 가진 배열을 만든다. 기본 자료형은 실수이므로 0., 1.처럼 출력된다.

3.2. `arange()`

`np.arange(1, 11, 2)`는 1부터 시작하여 2씩 증가하고 11 직전에서 멈춘다. Python의 `range()`와 마찬가지로 끝값은 포함하지 않는다.

3.3. `linspace()`

`np.linspace(0, 1, 5)`는 0부터 1까지의 구간을 균등하게 나누어 총 다섯 개의 값을 만든다. 끝값도 포함된다는 점이 `arange()`와 다르다.

3.4. 최종 코드: 02_numpy_array.py

```

1 import numpy as np
2
3 zeros = np.zeros(5)
4 ones = np.ones(5)
5 numbers = np.arange(1, 11, 2)
6 values = np.linspace(0, 1, 5)
7
8 print("Zeros :", zeros)
9 print("Ones :", ones)
10 print("Arange :", numbers)
11 print("Linspace :", values)

```

실행 결과는 다음과 같다.

```

Zeros : [0. 0. 0. 0. 0.]
Ones : [1. 1. 1. 1. 1.]
Arange : [1 3 5 7 9]
Linspace : [0.  0.25 0.5  0.75 1. ]

```

핵심 포인트

`arange()`는 간격을 기준으로 값을 만들고, `linspace()`는 전체 개수를 기준으로 값을 만든다. 그래프나 물리 시뮬레이션에서 일정 구간을 샘플링할 때는 `linspace()`가 자주 사용된다.

4. 인덱싱과 슬라이싱

NumPy 배열은 Python 리스트처럼 인덱싱과 슬라이싱을 지원한다. 인덱스는 0부터 시작하며 음수 인덱스로 뒤쪽 원소에 접근할 수 있다.

```
1 numbers[index]
2 numbers[start:end]
```

슬라이싱에서 시작 위치는 포함하고 끝 위치는 포함하지 않는다.

4.1. 최종 코드: 03_numpy_indexing.py

```
1 import numpy as np
2
3 numbers = np.array([10, 20, 30, 40, 50])
4
5 print(numbers[0])
6 print(numbers[2])
7 print(numbers[-1])
8
9 print(numbers[1:4])
10 print(numbers[:3])
11 print(numbers[2:])
```

실행 결과는 다음과 같다.

```
10
30
50
[20 30 40]
[10 20 30]
[30 40 50]
```

4.2. NumPy 슬라이싱의 View

Python 리스트를 슬라이싱하면 일반적으로 새로운 리스트가 만들어진다. 반면 NumPy의 슬라이싱 결과는 원본 배열의 일부를 바라보는 View인 경우가 많다. 따라서 슬라이스를 수정하면 원본 배열도 함께 바뀔 수 있다.

```
1 part = numbers[1:4]
2 part[0] = 999
```

원본과 완전히 독립된 배열이 필요할 때는 `copy()`를 사용한다.

```
1 part = numbers[1:4].copy()
```

핵심 포인트

NumPy 슬라이싱은 메모리를 절약하기 위해 원본 데이터를 공유할 수 있다. 원본 변경을 원하지 않는다면 `copy()`를 명시적으로 사용한다.

5. 벡터화 연산

Python 리스트에서 `+`는 리스트를 연결하지만, NumPy 배열에서 산술 연산자는 같은 위치의 원소끼리 계산한다. 이러한 방식을 벡터화 연산(vectorized operation)이라고 한다.

```
[1 2 3] + [4 5 6] = [5 7 9]
```

반복문을 직접 작성하지 않고 배열 전체를 한 번에 계산할 수 있으며, NumPy 내부의 최적화된 코드가 실행되므로 대량의 수치 계산에 효율적이다.

5.1. 최종 코드: 04_numpy_operations.py

```
1 import numpy as np
2
3 a = np.array([1, 2, 3])
4 b = np.array([4, 5, 6])
5
6 print(a + b)
7 print(a - b)
8 print(a * b)
9 print(a / b)
10
11 print()
12
13 print(a + 10)
14 print(a * 5)
```

실행 결과는 다음과 같다.

```
[5 7 9]
[-3 -3 -3]
[ 4 10 18]
[0.25 0.4  0.5 ]
```

```
[11 12 13]
[ 5 10 15]
```

5.2. 원소별 곱과 행렬곱

`a * b`는 같은 위치의 원소를 곱하는 원소별 곱이다. 선형대수의 행렬곱은 `@` 연산자를 사용한다.

연산	의미
<code>A * B</code>	원소별 곱
<code>A @ B</code>	행렬곱

핵심 포인트

NumPy에서는 가능한 한 Python 반복문 대신 배열 전체에 연산을 적용한다. 코드가 짧아지고 대량의 데이터 처리도 더 효율적이다.

6. 브로드캐스팅

브로드캐스팅(broadcasting)은 크기가 다른 배열 사이의 연산을 NumPy가 자동으로 확장하여 처리하는 기능이다.

예를 들어 배열과 하나의 숫자를 더하면 그 숫자가 모든 원소에 적용된다.

```
1 a = np.array([1, 2, 3])
2 print(a + 10)
```

개념적으로는 10이 [10, 10, 10]처럼 확장된 것으로 볼 수 있지만, NumPy는 실제로 같은 값을 반복 저장하지 않고 연산 규칙을 적용한다.

6.1. 행렬과 벡터의 브로드캐스팅

2차원 행렬의 각 행 길이와 1차원 벡터의 길이가 같다면 벡터를 각 행에 적용할 수 있다.

6.2. 최종 코드: 05_numpy_broadcasting.py

```
1 import numpy as np
2
3 a = np.array([1, 2, 3])
4 b = np.array([10])
5
6 print(a + b)
7
8 print()
9
10 matrix = np.array([[1, 2, 3],
11                   [4, 5, 6]])
12
13 vector = np.array([10, 20, 30])
14
15 print(matrix + vector)
```

실행 결과는 다음과 같다.

```
[11 12 13]
```

```
[[11 22 33]
 [14 25 36]]
```

행렬과 벡터의 계산은 개념적으로 다음과 같다.

```
[[1 2 3]      [[10 20 30]      [[11 22 33]
 [4 5 6]] +   [10 20 30]] =   [14 25 36]]
```

핵심 포인트

브로드캐스팅은 배열의 크기가 다르더라도 일정한 규칙에 따라 호환될 때만 가능하다. 호환되지 않는 모양의 배열을 연산하면 오류가 발생한다.

7. NumPy 선형대수

NumPy는 행렬곱, 전치행렬, 행렬식, 역행렬과 같은 기본 선형대수 연산을 지원한다.

표현	기능
A @ B	행렬곱
A.T	전치행렬
np.linalg.det(A)	행렬식
np.linalg.inv(A)	역행렬

7.1. 최종 코드: 06_numpy_linear_algebra.py

```

1 import numpy as np
2
3 A = np.array([[1, 2],
4               [3, 4]])
5
6 B = np.array([[5, 6],
7               [7, 8]])
8
9 print("Matrix Multiplication")
10 print(A @ B)
11
12 print()
13
14 print("Transpose")
15 print(A.T)
16
17 print()
18
19 print("Determinant")
20 print(np.linalg.det(A))
21
22 print()
23
24 print("Inverse")
25 print(np.linalg.inv(A))

```

실행 결과는 다음과 같다.

```

Matrix Multiplication
[[19 22]
 [43 50]]

```

```

Transpose
[[1 3]
 [2 4]]

```

```

Determinant
-2.0000000000000004

```

```

Inverse
[[-2.  1. ]
 [ 1.5 -0.5]]

```

부동소수점 계산에서는 수학적으로 -2 인 값이 -2.0000000000000004 처럼 출력될 수 있다. 이는 이진 부동소수점 표현에서 발생하는 정상적인 수치 오차이다.

7.2. 연립방정식 풀이

연립방정식 $Ax = b$ 를 풀 때 역행렬을 직접 계산하기보다 `np.linalg.solve(A, b)`를 사용하는 편이 일반적으로 더 효율적이고 수치적으로 안정적이다.

```

1 x = np.linalg.solve(A, b)

```


핵심 포인트

`np.linalg.inv()`는 역행렬이 존재하는 정사각행렬에만 사용할 수 있다. 실제 연립방정식 풀이에서는 역행렬을 직접 구하기보다 `solve()`를 우선 고려한다.

8. Day 7 종합 실습: NumPy 기초 복습

이번 종합 실습에서는 배열 생성, 인덱싱, 벡터화 연산, 브로드캐스팅, 선형대수 기능을 한 파일에서 복습한다.

8.1. 프로그램 요구사항

1. [10, 20, 30, 40, 50] 배열과 자료형을 출력한다.
2. `zeros()`, `ones()`, `arange()`, `linspace()`로 배열을 생성한다.
3. 지정된 배열에서 첫 원소, 마지막 원소, 세 번째 원소와 여러 슬라이스를 출력한다.
4. 두 배열의 덧셈, 뺄셈, 원소별 곱, 나눗셈과 스칼라 연산을 수행한다.
5. 2차원 행렬과 1차원 벡터를 브로드캐스팅하여 더한다.
6. 두 행렬의 행렬곱, 전치행렬, 행렬식, 역행렬을 출력한다.

8.2. 최종 코드: practice_day07.py

```

1 import numpy as np
2
3 # 1
4 v = np.array([10, 20, 30, 40, 50])
5 print(v)
6 print(type(v))
7
8 # 2
9 print(np.zeros(4))
10 print(np.ones(6))
11 print(np.arange(2, 12, 2))
12 print(np.linspace(0, 100, 5))
13
14 # 3
15 numbers = np.array([5, 10, 15, 20, 25])
16
17 print(numbers[0])
18 print(numbers[-1])
19 print(numbers[2])
20 print(numbers[:3])
21 print(numbers[-2:])
22 print(numbers[1:4])
23
24 # 4
25 a = np.array([2, 4, 6])
26 b = np.array([1, 3, 5])
27
28 print(a + b)
29 print(a - b)
30 print(a * b)
31 print(a / b)
32 print(a + 10)

```

```

33 print(b * 2)
34
35 # 5
36 matrix = np.array([
37     [1, 2, 3],
38     [4, 5, 6]
39 ])
40
41 vector = np.array([10, 20, 30])
42
43 print(matrix + vector)
44
45 # 6
46 A = np.array([
47     [2, 1],
48     [5, 3]
49 ])
50
51 B = np.array([
52     [1, 2],
53     [3, 4]
54 ])
55
56 print(A @ B)
57 print(A.T)
58 print(np.linalg.det(A))
59 print(np.linalg.inv(A))

```

8.3. 실행 결과

```

[10 20 30 40 50]
<class 'numpy.ndarray'>
[0. 0. 0. 0.]
[1. 1. 1. 1. 1. 1.]
[ 2  4  6  8 10]
[ 0. 25. 50. 75. 100.]
5
25
15
[ 5 10 15]
[20 25]
[10 15 20]
[ 3  7 11]
[1 1 1]
[ 2 12 30]
[2.          1.33333333 1.2          ]
[12 14 16]
[ 2  6 10]
[[11 22 33]
 [14 25 36]]
[[ 5  8]
 [14 22]]
[[2 5]
 [1 3]]
1.0000000000000009
[[ 3. -1.]

```

```
[-5. 2.]]
```

8.4. 프로그램 구조 분석

기능	사용한 개념
기본 배열 생성	<code>np.array()</code> 와 <code>ndarray</code>
규칙적인 배열 생성	<code>zeros()</code> , <code>ones()</code> , <code>arange()</code> , <code>linspace()</code>
일부 원소 선택	인덱싱과 슬라이싱
배열 전체 계산	벡터화 연산
행렬과 벡터 계산	브로드캐스팅
행렬곱 및 역행렬 계산	<code>np.linalg</code>

핵심 포인트

NumPy의 핵심은 배열을 만든 뒤 반복문 없이 배열 전체에 연산을 적용하는 것이다. 필요한 고급 기능은 실제 프로젝트나 전공 문제에서 사용할 때 추가로 학습하면 된다.

9. 실습 파일 목록

실습

Day 7의 학습 파일은 다음과 같이 관리한다.

- 01_numpy_intro.py
- 02_numpy_array.py
- 03_numpy_indexing.py
- 04_numpy_operations.py
- 05_numpy_broadcasting.py
- 06_numpy_linear_algebra.py
- practice_day07.py
- python_day07.tex
- python_day07.pdf

10. 핵심 요약

- NumPy는 Python에서 효율적인 수치 계산을 제공하는 라이브러리이다.
- NumPy의 기본 배열 자료형은 `ndarray`이다.
- `np.array()`는 Python 리스트를 NumPy 배열로 변환한다.
- `zeros()`와 `ones()`는 일정한 값으로 채운 배열을 생성한다.
- `arange()`는 간격을 기준으로, `linspace()`는 개수를 기준으로 배열을 생성한다.
- NumPy 배열은 Python 리스트처럼 인덱싱과 슬라이싱을 지원한다.
- NumPy 슬라이싱은 원본 데이터를 공유하는 View일 수 있다.

- 배열 연산은 같은 위치의 원소끼리 계산되는 벡터화 연산이다.
- 배열과 하나의 숫자를 연산하면 그 숫자가 모든 원소에 적용된다.
- 브로드캐스팅은 호환되는 서로 다른 크기의 배열을 자동으로 확장하여 계산한다.
- *는 원소별 곱이고 @는 행렬곱이다.
- .T는 전치행렬을 반환한다.
- `np.linalg.det()`는 행렬식, `np.linalg.inv()`는 역행렬을 계산한다.
- 실제 연립방정식 풀이에는 `np.linalg.solve()`가 더 적합한 경우가 많다.
- NumPy의 고급 기능은 프로젝트와 전공 문제에서 필요한 시점에 추가 학습한다.